Answers

How to check yourself

Score 1 point for each bug you found and 1 for each fix that works; drills D1 to D16 hide 45 bugs in total.

A bug counts only if you named the input that breaks it and what happens: an exception, or a wrong value. Rough guide: 40 or more means this layer is close to automatic. Below that, reread the study patterns listed under each drill you missed, then redo the drill three days later.

Every buggy result and every fix below was run on Java 21 before it was written down.

D1 answer

The bug: `hashCode()` uses `balance`, a field that `equals()` ignores and that changes, so `contains(acc)` returns false after the deposit.

Two reasons it's wrong:

1. Contract break: two accounts with the same number but different balances are `equals()` but have different hash codes.
2. Mutable hash: `deposit()` changes the hash after insertion, so the set looks in the wrong bucket.

Fix:

```
@Override
public int hashCode() {
    return Objects.hash(accountNumber);   // the same field as equals(), and
it never changes
}
```

Also make `equals()` null-safe with `Objects.equals(accountNumber, other.accountNumber)`.

D2 answer

The bug: `getOrDefault()` returns a new list for a missing key but never stores it, so every `add()` goes into a list nobody keeps and the map stays empty.

Fix:

```
Map<String, List<Payment>> bySender(List<Payment> payments) {
    Map<String, List<Payment>> result = new HashMap<>();
    for (Payment p : payments) {
        result.computeIfAbsent(p.getFromAccount(), k -> new ArrayList<>
()).add(p);
    }
    return result;
}
// Stream version:
payments.stream().collect(Collectors.groupingBy(Payment::getFromAccount))
```

Extension:

```
Map<String, Integer> countBySender(List<Payment> payments) {
    Map<String, Integer> counts = new HashMap<>();
    for (Payment p : payments) {
        counts.merge(p.getFromAccount(), 1, Integer::sum);
    }
    return counts;
}
```

Edge cases: an empty list, a single sender, a null `fromAccount` (fine in a `HashMap` loop, but `groupingBy` would throw).

D3 answer

Bugs:

1. No status filter, so REJECTED and PENDING payments are counted.
2. The merge function `(a, b) -> a` keeps the first amount instead of adding: USD 100 + 25 comes out as 100.
3. `toMap()` builds a `HashMap`, which isn't sorted by currency.

Fix:

```
Map<String, Double> totalByCurrency(List<Payment> payments) {
    return payments.stream()
            .filter(p -> "COMPLETED".equals(p.getStatus()))
            .filter(p -> p.getCurrency() != null)        // a TreeMap rejects
null keys
            .collect(Collectors.toMap(Payment::getCurrency,
Payment::getAmount, Double::sum, TreeMap::new));
}
```

Alternative: `Collectors.groupingBy(Payment::getCurrency, TreeMap::new, Collectors.summingDouble(Payment::getAmount))`.

Edge cases: a currency seen once, a null currency (`TreeMap` throws `NullPointerException`), nothing COMPLETED.

D4 answer

Bugs:

1. `> 10_000` excludes exactly 10,000, but the rule says "10,000 or more", so it must be `>=`.
2. `groupingBy()` only creates keys that occur. On a day with no high-value payments there is no `true` key, so `counts.get(true)` returns null and unboxing it throws `NullPointerException`.

Fix:

```
Map<Boolean, Long> counts = payments.stream()
        .collect(Collectors.partitioningBy(p -> p.getAmount() >= 10_000,
Collectors.counting()));

long highValue = counts.get(true);    // partitioningBy always has both keys
long normal = counts.get(false);
```

Edge cases: exactly 10,000, an empty list, every payment on one side.

D5 answer

Bugs:

1. `p.getStatus() == "COMPLETED"` compares references. It passes with string literals in tests, but statuses read from a database or JSON are new objects, so real COMPLETED payments are skipped.
2. `p.getLabels()` can be null, so `flatMap` throws `NullPointerException`.
3. A `Set` has no order, so the result isn't sorted A to Z.

Fix:

```
List<String> labelsOfCompleted(List<Payment> payments) {
    return payments.stream()
            .filter(p -> "COMPLETED".equals(p.getStatus()))
            .map(Payment::getLabels)
            .filter(Objects::nonNull)
            .flatMap(List::stream)
            .distinct()
            .sorted()
            .collect(Collectors.toList());
}
```

Edge cases: null labels, an empty label list, the same label on two payments, a status built with `new String("COMPLETED")`.

D6 answer

Bugs:

1. `count++` sits outside the `if`, so it counts every payment: ACC-1 sending 100 and 200 gives 300 / 3 = 100 instead of 150.
2. With no matching payments (or an empty list) it returns 0.0 / 0, which is `NaN`, not the required 0.0.
3. `p.getFromAccount().equals(account)` and `p.getStatus().equals(...)` throw `NullPointerException` when a field is null.

Fix:

```
double averageSent(List<Payment> payments, String account) {
    return payments.stream()
            .filter(p -> Objects.equals(account, p.getFromAccount()))
            .filter(p -> "COMPLETED".equals(p.getStatus()))
            .mapToDouble(Payment::getAmount)
            .average()
            .orElse(0.0);
}
```

Edge cases: no payments from the account, only REJECTED ones, a null `fromAccount`.

D7 answer

Bugs:

1. `max()` on `createdAt` returns the newest payment; the oldest is `min()`.
2. `get()` on an empty `Optional` throws `NoSuchElementException: No value present`, not the required `IllegalStateException` with a clear message.

Fix:

```
String oldestPendingId(List<Payment> payments) {
    return payments.stream()
            .filter(p -> "PENDING".equals(p.getStatus()))
            .min(Comparator.comparing(Payment::getCreatedAt))
            .map(Payment::getId)
            .orElseThrow(() -> new IllegalStateException("No pending
payments"));
}
```

Edge cases: no PENDING payments, two with the same `createdAt` (add `.thenComparing(Payment::getId)`), a null `createdAt` (use `Comparator.comparing(Payment::getCreatedAt, Comparator.nullsLast(Comparator.naturalOrder()))`).

D8 answer

Bugs:

1. Alphabetical status order (COMPLETED, PENDING, REJECTED) doesn't put PENDING first.
2. The trailing `.reversed()` flips every field: amounts happen to come out highest first, but statuses run Z to A and ids come out Z to A (P5 before P1).

Buggy result for the hint data: P4 REJECTED 900, P2 PENDING 500, P3 PENDING 100, P5 COMPLETED 100, P1 COMPLETED 100.

Fix:

```
payments.sort(Comparator
        .comparingInt((Payment p) -> "PENDING".equals(p.getStatus()) ? 0 : 1)

.thenComparing(Comparator.comparingDouble(Payment::getAmount).reversed())
        .thenComparing(Payment::getId));
// P2 PENDING 500, P3 PENDING 100, P4 REJECTED 900, P1 COMPLETED 100, P5
COMPLETED 100
```

The lambda needs its type, `(Payment p)`, because it starts a chain (study pattern 14).

Edge cases: a null status (lands in the second group here), equal amounts, a null id.

D9 answer

Bugs:

1. It returns `null` for an empty or null list, which breaks the "never null" rule and every caller that loops over the result.
2. `p.getLabels()` can be null, so `.contains()` throws `NullPointerException`.
3. A null element in the list throws on `p.getLabels()`.

Fix:

```
List<String> urgentIds(List<Payment> payments) {
    if (payments == null) {
        return new ArrayList<>();
    }
    return payments.stream()
            .filter(Objects::nonNull)
            .filter(p -> p.getLabels() != null &&
p.getLabels().contains("urgent"))
            .map(Payment::getId)
            .collect(Collectors.toList());
}
```

Edge cases: a null list, an empty list, a payment with no labels, "URGENT" in upper case (ask whether labels are case-sensitive).

D10 answer

Bugs:

1. `putIfAbsent()` keeps the first attempt (retry 0), not the latest.
2. A `HashMap` doesn't keep first-seen order; the buggy result came out P2, P7 (retry 0), P9.

Fix:

```
List<Payment> latestAttempts(List<Payment> payments) {
    Map<String, Payment> byId = new LinkedHashMap<>();    // keeps the first-
seen order of ids
    for (Payment p : payments) {
        byId.merge(p.getId(), p,
                (current, candidate) -> candidate.getRetryCount() >
current.getRetryCount() ? candidate : current);
    }
    return new ArrayList<>(byId.values());
}
// P7 (retry 2), P2, P9
```

Replacing a value in a `LinkedHashMap` doesn't move its key, so the first-seen order survives every merge.

Edge cases: retries arriving out of order (retry 2 before retry 1), two rows with the same retry count (decide which wins), a single id.

D11 answer

Bugs:

1. Pages start at 1, so `from` must be `(pageNumber - 1) * pageSize`; page 1 currently returns P3 and P4.
2. `to` can run past the end: page 3 throws `IndexOutOfBoundsException: toIndex = 8` instead of returning [P5].
3. A page past the end must return an empty list, not throw.
4. No validation: page 0, negative values and a page size of 0 are all accepted.

Bonus point: `subList()` returns a live view of the caller's list, not a copy (study pattern 25).

Fix:

```
List<Payment> page(List<Payment> payments, int pageNumber, int pageSize) {
    if (pageNumber < 1 || pageSize < 1) {
        throw new IllegalArgumentException("pageNumber and pageSize must be
at least 1");
    }
    long from = (long) (pageNumber - 1) * pageSize;   // long: huge inputs
can't overflow
    if (from >= payments.size()) {
        return new ArrayList<>();
    }
    int to = (int) Math.min(from + pageSize, payments.size());
    return new ArrayList<>(payments.subList((int) from, to));
}
// pages 1 to 4 with size 2: [P1, P2], [P3, P4], [P5], []
```

Edge cases: the last partial page, exactly one full page, a page past the end, a page size bigger than the list, very large numbers (in `int`, `(pageNumber - 1) * pageSize` can overflow into a negative index).

D12 answer

Bugs:

1. `&&` binds tighter than `||`, so the code means REJECTED OR (PENDING and under the limit and recent). A REJECTED payment with 10 retries from 5 days ago returns true: an endless retry loop.

2. `retryCount <= 3` allows a fourth retry; the rule says less than 3.
3. `p.getStatus().equals(...)` throws on a null status, and a null `createdAt` throws as well.

Fix:

```
boolean eligibleForRetry(Payment p, LocalDateTime now) {
    boolean retryableStatus = "REJECTED".equals(p.getStatus()) ||
"PENDING".equals(p.getStatus());
    boolean underLimit = p.getRetryCount() < 3;
    boolean recent = p.getCreatedAt() != null &&
p.getCreatedAt().isAfter(now.minusHours(24));
    return retryableStatus && underLimit && recent;
}
```

Named booleans make each rule readable and testable on its own. Ask the interviewer: does a payment created exactly 24 hours ago still count as "within the last 24 hours"?

Edge cases: `retryCount` 2 and 3, exactly 24 hours old, COMPLETED with 0 retries, a null status.

D13 answer

Bugs:

1. `getPayments()` returns the internal list, so any caller can add, remove or `clear()` the service's data.
2. `biggestFive()` sorts the service's own list, changing the order every other reader sees (and racing with other threads).
3. `(int) (b.getAmount() - a.getAmount())` treats amounts less than 1 apart as equal: 10.75 vs 10.25 gives 0.
4. `subList(0, 5)` throws `IndexOutOfBoundsException` with fewer than 5 payments, and returns a live view rather than a copy.

Fix:

```
class PaymentService {
    private final List<Payment> payments = new ArrayList<>();

    public List<Payment> getPayments() {
        return Collections.unmodifiableList(payments);    // read-only view
    }

    public List<Payment> biggestFive() {
        return payments.stream()

.sorted(Comparator.comparingDouble(Payment::getAmount).reversed())
                .limit(5)
                .collect(Collectors.toList());            // a new list;
nothing internal changes
    }
}
```

Edge cases: fewer than 5 payments, amounts a few pence apart, a caller calling `clear()` on the returned list (it now throws `UnsupportedOperationException`).

D14 answer

Bugs:

1. The `try` wraps the whole loop, so the first failure stops every payment after it. With P2 failing, P3 is never sent and the method returns 1.
2. `catch (Exception e)` also swallows real bugs such as `NullPointerException`.
3. Nothing records which payment failed or why; printing "error" loses the id, the message and the stack trace.

Fix, with the extension:

```
class BatchResult {
    final List<String> succeeded = new ArrayList<>();
    final Map<String, String> failed = new LinkedHashMap<>();   // payment id
-> reason
}

BatchResult processAll(List<Payment> payments, PaymentGateway gateway) {
    BatchResult result = new BatchResult();
    for (Payment p : payments) {
        try {
            gateway.send(p);
            result.succeeded.add(p.getId());
        } catch (GatewayException e) {                          // only the
failure we expect
            log.warn("Payment {} failed: {}", p.getId(), e.getMessage(), e);
            result.failed.put(p.getId(), e.getMessage());
        }
    }
    return result;
}
// succeeded = [P1, P3], failed = {P2=Insufficient funds}
```

`log` is an SLF4J-style logger. Two follow-ups interviewers like: should an unexpected `RuntimeException` stop the batch or be recorded too? And if the whole batch is retried, how do you avoid sending P1 twice? (Idempotency keys, a Level 4 topic.)

D15 answer

Bugs:

1. `payments.remove(p)` inside a for-each loop throws `ConcurrentModificationException` (or silently skips an item).
2. `new BigDecimal(p.getAmount())` builds from a `double`: 250.0 becomes 250, and amounts like 0.1 become long binary fractions.
3. `BigDecimal.equals()` compares scale, so 250 and 250.00 count as different and matching payments get removed.
4. `ledger.get(...)` returns null for a payment missing from the ledger, so `ledgerAmount.equals(...)` throws `NullPointerException`.
5. `Integer == Integer` compares references: with a few hundred payments, 200 == 200 is false, so "Reconciled" never prints.

Fix:

```
void reconcile(List<Payment> payments, Map<String, BigDecimal> ledger,
Map<String, Integer> expected) {
    payments.removeIf(p -> {
        BigDecimal ledgerAmount = ledger.get(p.getId());
        return ledgerAmount == null
// not in the ledger: a mismatch (confirm)
                || ledgerAmount.compareTo(BigDecimal.valueOf(p.getAmount()))
!= 0;
    });
    if (Objects.equals(payments.size(), expected.get("count"))) {
        System.out.println("Reconciled");
    }
}
```

Ask: should payments missing from the ledger be removed, kept, or reported? Better still, store payment amounts as `BigDecimal` from the start.

Edge cases: more than 127 payments, an amount like 0.10, a payment missing from the ledger, a missing "count" key.

D16 answer

Bugs:

1. Day 30 doesn't exist in February: `LocalDateTime.of(2026, 2, 30, ...)` throws `DateTimeException`.
2. 31-day months lose the 31st.
3. `isAfter(start)` is strict, so a payment at exactly 00:00 on the 1st is excluded.
4. `isBefore(... 23:59)` drops everything in the last minute, such as 23:59:30.

For October the buggy code keeps only the 15th, out of payments on 1 Oct 00:00, 15 Oct, 30 Oct 23:59:30 and 31 Oct.

Fix:

```
List<Payment> inMonth(List<Payment> payments, int year, int month) {
    YearMonth target = YearMonth.of(year, month);
    LocalDateTime start = target.atDay(1).atStartOfDay();
    LocalDateTime nextMonthStart =
target.plusMonths(1).atDay(1).atStartOfDay();
    return payments.stream()
            .filter(p -> p.getCreatedAt() != null)
            .filter(p -> !p.getCreatedAt().isBefore(start) &&
p.getCreatedAt().isBefore(nextMonthStart))
            .collect(Collectors.toList());
}
// Simpler: .filter(p -> YearMonth.from(p.getCreatedAt()).equals(target))
```

Edge cases: 1 October 00:00, 31 October, 1 November 00:00 (excluded), February in a leap year, payments recorded in a different time zone.

K1 answer

Part 1 bugs:

1. The signs are swapped: money sent (`fromAccount`) must be subtracted and money received added. The buggy code returns 1,075 instead of 925.
2. `largestOutgoing` starts from `payments.get(0)`, which may be another account's payment (or a REJECTED one), so a stranger's 5,000 payment can win.
3. `i <= payments.size()` reads one past the end, so the method throws `IndexOutOfBoundsException` on every non-empty list.
4. An empty list throws on `get(0)`, and an account with no outgoing payments has no honest answer: return `Optional`.
5. `largestOutgoing` ignores the COMPLETED rule.
6. `p.getStatus().equals(...)` and `p.getFromAccount().equals(...)` throw on null fields; put the known value on the left.

Fixed:

```
public double closingBalance(double opening, List<Payment> payments, String
account) {
    double balance = opening;
    for (Payment p : payments) {
        if (p == null || !"COMPLETED".equals(p.getStatus())) {
            continue;
        }
        if (account.equals(p.getToAccount())) {
            balance += p.getAmount();      // money in
        }
```

```
        if (account.equals(p.getFromAccount())) {
            balance -= p.getAmount();      // money out
        }
    }
    return balance;
}

public Optional<Payment> largestOutgoing(List<Payment> payments, String
account) {
    return payments.stream()
            .filter(Objects::nonNull)
            .filter(p -> "COMPLETED".equals(p.getStatus()))
            .filter(p -> account.equals(p.getFromAccount()))
            .max(Comparator.comparingDouble(Payment::getAmount));
}
```

Part 2:

```
public Map<String, Double> netFlowByCounterparty(List<Payment> payments,
String account) {
    Map<String, Double> net = new TreeMap<>();                          //
sorted by account
    for (Payment p : payments) {
        if (p == null || !"COMPLETED".equals(p.getStatus())) {
            continue;
        }
        if (account.equals(p.getToAccount()) && p.getFromAccount() != null) {
            net.merge(p.getFromAccount(), p.getAmount(), Double::sum);   //
they paid us
        }
        if (account.equals(p.getFromAccount()) && p.getToAccount() != null) {
            net.merge(p.getToAccount(), -p.getAmount(), Double::sum);    //
we paid them
        }
    }
    return net;
}
// {ACC-2=-125.0, ACC-3=50.0}

public List<String> accountsWithRepeatedRejections(List<Payment> payments) {
    Map<String, Map<LocalDate, Long>> rejectionsByAccountAndDay =
payments.stream()
            .filter(Objects::nonNull)
            .filter(p -> "REJECTED".equals(p.getStatus()))
            .filter(p -> p.getFromAccount() != null && p.getCreatedAt() !=
null)
```

```
            .collect(Collectors.groupingBy(
                    Payment::getFromAccount,
                    Collectors.groupingBy(p ->
p.getCreatedAt().toLocalDate(), Collectors.counting())));
    return rejectionsByAccountAndDay.entrySet().stream()
            .filter(e -> e.getValue().values().stream().anyMatch(count ->
count >= 3))
            .map(Map.Entry::getKey)
            .sorted()
            .collect(Collectors.toList());
}
```

The nested map (account, then day, then count) avoids string keys like account + date, which can collide (study pattern 27).

Part 3:

Method	Time	Extra space
`closingBalance`	O(n)	O(1)
`largestOutgoing`	O(n)	O(1)
`netFlowByCounterparty`	O(n log c) for c counterparties (TreeMap)	O(c)
`accountsWithRepeatedRejections`	O(n + a log a) for a accounts	O(n) worst case

Edge cases to say out loud: an empty list, no COMPLETED payments, a payment from the account to itself (it adds and subtracts the same amount), rejections at 23:59 and 00:01 on different days, exactly 3 rejections, a null `createdAt`.

K2 answer

Part 1 bugs:

1. `spentToday.get(account)` is null for an account's first payment, and unboxing it into a `double` throws `NullPointerException`.
2. `<` rejects a payment that brings the total to exactly 5,000; the limit allows it, so use `<=`.
3. Zero and negative amounts are accepted, and a payment of -1,000 would raise the remaining limit.
4. "Today" never resets: the map grows forever, and yesterday's spending blocks today's payments.

5. Not thread-safe: two requests can both read the same `spent`, both pass the check, and together exceed the limit (a check-then-act race).

Also worth saying: real money should be `BigDecimal`, and the clock should be injected so tests can control "today".

Fixed, with both extensions:

```
class LimitChecker {
    private static final double DAILY_LIMIT = 5000;
    private final Map<String, Double> spentToday = new HashMap<>();
    private final Clock clock;              // injected, so tests can move
time
    private LocalDate currentDay;

    LimitChecker(Clock clock) {
        this.clock = clock;
    }

    public synchronized boolean tryPay(String account, double amount) {
        if (account == null || amount <= 0) {
            throw new IllegalArgumentException("account is required and
amount must be positive");
        }
        resetIfNewDay();
        double spent = spentToday.getOrDefault(account, 0.0);
        if (spent + amount <= DAILY_LIMIT) {
            spentToday.put(account, spent + amount);
            return true;
        }
        return false;
    }

    // Extension 1
    public synchronized double remainingToday(String account) {
        resetIfNewDay();
        return DAILY_LIMIT - spentToday.getOrDefault(account, 0.0);
    }

    // Extension 2
    public synchronized List<String> accountsNearLimit(double percent) {
        resetIfNewDay();
        double threshold = DAILY_LIMIT * percent / 100;
        return spentToday.entrySet().stream()
                .filter(e -> e.getValue() >= threshold)
                .sorted(Map.Entry.<String,
Double>comparingByValue().reversed())
```

```
                .map(Map.Entry::getKey)
                .collect(Collectors.toList());
    }

    private void resetIfNewDay() {
        LocalDate today = LocalDate.now(clock);
        if (!today.equals(currentDay)) {
            spentToday.clear();
            currentDay = today;
        }
    }
}
```

Checked behaviour: 3,000 then 2,000 is approved (exactly the limit), 0.01 more is refused, -100 throws, and on the next day the counter starts again from zero. `tryPay` and `remainingToday` are O(1); `accountsNearLimit` is O(a log a) for a accounts.

Part 3, three servers:

- Each server keeps its own in-memory map, so one account can spend up to 5,000 on each server: 15,000 a day.
- A restart or deployment wipes every counter.
- `synchronized` only protects one JVM, not three.

The fix is to keep the counter in shared storage and make check-and-add one atomic step, for example a single conditional SQL update:

```
UPDATE daily_spend
SET spent = spent + :amount
WHERE account_id = :account
  AND day = :today
  AND spent + :amount <= 5000;
-- 1 row updated: approved. 0 rows: refused (create the day's row first if it
doesn't exist).
```

Alternatives are a row lock (`SELECT ... FOR UPDATE` inside a transaction) or an atomic check-and-increment in Redis. Also make payments idempotent, so a retried request isn't counted twice.